

Национальный исследовательский университет ИТМО

Факультет безопасности информационных технологий

## Алгоритмы и структуры данных

Лабораторная работа №3

«Цифровая сортировка дека с поиском методом  
Райта»

**Выполнил:**

Студент группы N3247

Сивоконь А.А.

---

(подпись)

**Преподаватель:**

Ерофеев С.А.

---

(отметка о выполнении)

---

(подпись)

Санкт-Петербург

2026

# Содержание

|          |                                                  |           |
|----------|--------------------------------------------------|-----------|
| <b>1</b> | <b>Постановка задачи</b>                         | <b>2</b>  |
| <b>2</b> | <b>Описание структур данных и алгоритмов</b>     | <b>3</b>  |
| 2.1      | Дек на двусвязном списке . . . . .               | 3         |
| 2.2      | Алгоритм цифровой сортировки LSD . . . . .       | 3         |
| 2.3      | Поиск методом Райта . . . . .                    | 4         |
| 2.4      | Маршрут программы . . . . .                      | 5         |
| <b>3</b> | <b>Описание используемых переменных</b>          | <b>6</b>  |
| <b>4</b> | <b>Тестирование программы</b>                    | <b>8</b>  |
| 4.1      | Тест 1: стандартная сортировка и поиск . . . . . | 8         |
| 4.2      | Тест 2: дек с отрицательными числами . . . . .   | 8         |
| 4.3      | Тест 3: дек из одного элемента . . . . .         | 9         |
| 4.4      | Тест 4: несуществующий файл . . . . .            | 9         |
| 4.5      | Тест 5: пустой файл . . . . .                    | 10        |
| <b>5</b> | <b>Блок-схемы алгоритма</b>                      | <b>11</b> |
| <b>6</b> | <b>Исходный код программы</b>                    | <b>12</b> |
| 6.1      | Файл deque.h . . . . .                           | 12        |
| 6.2      | Файл deque.c . . . . .                           | 13        |
| 6.3      | Файл main.c . . . . .                            | 19        |
|          | <b>Заключение</b>                                | <b>22</b> |
|          | <b>Приложение А. Блок-схемы алгоритма</b>        | <b>23</b> |

# 1. Постановка задачи

**Цель работы:** разработать программу цифровой сортировки (Radix Sort LSD) для последовательности целых чисел (включая отрицательные), считываемых из файла, используя в качестве основной и вспомогательной структуры данных **дек на основе двусвязного списка**; реализовать **поиск методом Райта** в отсортированном деке; определить сложность программы.

**Для достижения поставленной цели необходимо решить следующие задачи:**

1. Реализовать дек на двусвязном списке с операциями `push_front`, `push_back`, `pop_front`, `pop_back`, `deque_is_empty`, `deque_free`;
2. Реализовать чтение последовательности целых чисел из файла непосредственно в дек;
3. Реализовать алгоритм цифровой сортировки LSD через 10 деков-корзин;
4. Корректно обработать отрицательные числа: разделить дек на два вспомогательных, отсортировать каждый независимо, объединить результат;
5. Реализовать поиск методом Райта в отсортированном деке;
6. Оценить теоретическую сложность алгоритмов;
7. Провести тестирование программы на различных наборах данных;
8. Составить блок-схемы алгоритма и оформить отчёт.

## 2. Описание структур данных и алгоритмов

### 2.1. Дек на двусвязном списке

Основной структурой данных является дек (двусторонняя очередь) на основе динамически выделяемых узлов двусвязного списка:

- Узел `DNode` содержит значение `value` и два указателя: `prev` (на предыдущий узел) и `next` (на следующий).
- Дек `Deque` хранит указатели `front` (голова) и `back` (хвост), а также счётчик `size`. Двусвязность обеспечивает  $O(1)$  для вставки и удаления *с обоих концов*, что необходимо как для сортировки, так и для поиска методом Райта.
- При добавлении первого элемента он становится одновременно головой и хвостом: оба указателя `front` и `back` ссылаются на него.

Операции дека:

| Функция                     | Описание                                                                 |
|-----------------------------|--------------------------------------------------------------------------|
| <code>deque_init</code>     | обнулить поля <code>front</code> , <code>back</code> , <code>size</code> |
| <code>deque_is_empty</code> | вернуть <code>size==0</code>                                             |
| <code>push_front</code>     | добавить элемент в голову                                                |
| <code>push_back</code>      | добавить элемент в хвост                                                 |
| <code>pop_front</code>      | извлечь элемент из головы                                                |
| <code>pop_back</code>       | извлечь элемент из хвоста                                                |
| <code>deque_free</code>     | освободить все узлы                                                      |

### 2.2. Алгоритм цифровой сортировки LSD

Цифровая сортировка — нерекурсивный алгоритм, обрабатывающий числа поразрядно от *младшего* разряда к *старшему* (LSD — Least Significant Digit).

Пошаговый алгоритм функции `counting_sort_by_deque`:

1. **Шаг 1.** Инициализировать 10 деков-корзин `buckets[0..9]`.

2. **Шаг 2.** Пока исходный дек не пуст: извлечь элемент через `pop_front`, вычислить цифру текущего разряда  $(v / \text{exp}) \% 10$ , добавить элемент в `buckets[digit]` через `push_back`.
3. **Шаг 3.** Обходя корзины `buckets[0..9]` по порядку: пока корзина не пуста, извлекать элементы через `pop_front` и добавлять обратно в исходный дек через `push_back`. Порядок FIFO (добавление в хвост, извлечение из головы) гарантирует устойчивость сортировки.

**Поддержка отрицательных чисел.** Функция `radix_sort` разделяет исходный дек на два вспомогательных: `neg` — модули отрицательных чисел, `pos` — неотрицательные числа. Каждый дек сортируется независимо через `radix_sort_nonneg`. При сборе результата из `neg` элементы извлекаются с *хвоста* (`pop_back`) — потому что наибольший модуль соответствует наименьшему отрицательному числу, — и добавляются со знаком минус. Затем добавляются элементы из `pos` с головы (`pop_front`).

## 2.3. Поиск методом Райта

Поиск методом Райта предназначен для работы с *упорядоченным* двусвязным списком и позволяет в среднем вдвое сократить число сравнений по сравнению с линейным поиском от головы.

**Алгоритм функции `wright_search`:**

1. **Шаг 1. Выход на середину.** От головы дека выполнить  $\lfloor n/2 \rfloor$  шагов по указателю `next` — получить указатель на средний элемент. На этом шаге сравнения с ключом не производятся.
2. **Шаг 2. Сравнение с серединой.** Если `key == cur->value` — элемент найден.
3. **Шаг 3. Направленное движение.** Если `key < cur->value` — двигаться по `prev` влево, пока `cur->value > key`. Иначе — двигаться по `next` вправо, пока `cur->value < key`. Условие раннего выхода следует из упорядоченности: ключ заведомо отсутствует в оставшейся части.

4. **Шаг 4. Проверка точки останова.** Если остановились на узле с `cur->value == key` — найден; иначе — отсутствует.

**Почему необходим двусвязный список.** Шаг 3 требует движения от середины как влево (по `prev`), так и вправо (по `next`). Односвязный список не позволяет двигаться назад, поэтому метод Райта применим только к двусвязной структуре.

## 2.4. Маршрут программы

1. **Чтение входного файла (`read_file`):** однократное считывание чисел из файла непосредственно в дек через `push_back`.
2. **Вывод исходного дека (`deque_print`).**
3. **Выполнение Radix Sort (`radix_sort`):** разделение на отрицательные/неотрицательные, сортировка, объединение.
4. **Вывод отсортированного дека (`deque_print`).**
5. **Поиск методом Райта (`wright_search`):** ключи задаются аргументами командной строки или вводятся с клавиатуры.
6. **Освобождение памяти:** `deque_free`.

### 3. Описание используемых переменных

В таблице 1 приведено описание переменных программы.

Таблица 1 – Описание переменных программы

| Переменная                        | Тип          | Границы           | Назначение                   |
|-----------------------------------|--------------|-------------------|------------------------------|
| <b>Главная программа (main.c)</b> |              |                   |                              |
| filename                          | const char*  | строка            | Имя входного файла           |
| d                                 | Deque        | структура         | Основной дек программы       |
| n                                 | int          | $[1; 2^{31} - 1]$ | Количество прочитанных чисел |
| key                               | int          | любое целое       | Ключ поиска                  |
| pos                               | int          | $[-1; n - 1]$     | Позиция найденного элемента  |
| views                             | int          | $[0; n]$          | Число просмотров при поиске  |
| node                              | const DNode* | адрес / NULL      | Результат поиска<br>Райта    |
| <b>Узел и дек (deque.c)</b>       |              |                   |                              |
| DNode.value                       | int          | любое целое       | Хранимое значение            |
| DNode.prev                        | DNode*       | адрес / NULL      | Указатель на предыдущий узел |
| DNode.next                        | DNode*       | адрес / NULL      | Указатель на следующий узел  |
| Deque.front                       | DNode*       | адрес / NULL      | Голова дека                  |
| Deque.back                        | DNode*       | адрес / NULL      | Хвост дека                   |
| Deque.size                        | int          | $[0; n]$          | Текущий размер дека          |
| cur                               | DNode*       | адрес / NULL      | Текущий узел при обходе      |
| <b>Ввод из файла (deque.c)</b>    |              |                   |                              |

| Переменная                           | Тип          | Границы                      | Назначение                      |
|--------------------------------------|--------------|------------------------------|---------------------------------|
| fp                                   | FILE*        | адрес                        | Указатель на открытый файл      |
| tmp                                  | int          | любое целое                  | Очередное считанное число       |
| <b>Цифровая сортировка (deque.c)</b> |              |                              |                                 |
| exp                                  | int          | $\{1, 10, \dots, 10^{d-1}\}$ | Текущий разряд                  |
| buckets[10]                          | Deque[10]    | —                            | Десять деков-корзин             |
| digit                                | int          | $[0; 9]$                     | Цифра текущего разряда          |
| max_val                              | int          | $[0; 2^{31} - 1]$            | Максимум для определения $d$    |
| neg                                  | Deque        | структура                    | Дек модулей отрицательных чисел |
| pos                                  | Deque        | структура                    | Дек неотрицательных чисел       |
| value                                | int          | любое целое                  | Временная при pop/push          |
| <b>Поиск методом Райта (deque.c)</b> |              |                              |                                 |
| mid                                  | int          | $[0; n/2]$                   | Индекс среднего элемента        |
| idx                                  | int          | $[0; n - 1]$                 | Текущая позиция при обходе      |
| cur                                  | const DNode* | адрес / NULL                 | Текущий узел при поиске         |



## 4. Тестирование программы

### 4.1. Тест 1: стандартная сортировка и поиск

Файл `input.txt`: 170 45 75 90 802 24 2 66 -73 -4

```
$ ./sort input.txt 75 2 802

Чтение файла 'input.txt'...
Прочитано 10 чисел.

Исходный дек:
170 45 75 90 802 24 2 66 -73 -4

Выполняем цифровую сортировку дека...
Отсортированный дек:
-73 -4 2 24 45 66 75 90 170 802

Поиск методом Райта
Ключ 75: НАЙДЕН на позиции 6 (просмотров: 2)
Ключ 2: НАЙДЕН на позиции 2 (просмотров: 4)
Ключ 802: НАЙДЕН на позиции 9 (просмотров: 5)
```

Дек успешно отсортирован. Все три ключа найдены. Для ключа 75 (позиция 6): средний элемент на позиции 5 равен  $66 < 75$ , поэтому поиск сдвигается вправо — найдено за 2 просмотра.

### 4.2. Тест 2: дек с отрицательными числами

Входной файл `neg.txt`: -15 3 -7 42 -2 18

```
$ ./sort neg.txt -15 5

Чтение файла 'neg.txt'...
Прочитано 6 чисел.

Исходный дек:
-15 3 -7 42 -2 18

Выполняем цифровую сортировку дека...
Отсортированный дек:
-15 -7 -2 3 18 42

Поиск методом Райта
```

```
Ключ -15: НАЙДЕН на позиции 0 (просмотров: 4)
Ключ 5: не найден (просмотров: 2)
```

Программа корректно разделила дек на  $\text{neg} = \{15, 7, 2\}$  (по модулю) и  $\text{pos} = \{3, 42, 18\}$ , отсортировала каждый и собрала результат: сначала отрицательные в порядке возрастания  $(-15, -7, -2)$ , затем положительные  $(3, 18, 42)$ .

### 4.3. Тест 3: дек из одного элемента

```
$ ./sort one.txt 5 9

Чтение файла 'one.txt'...
Прочитано 1 число.

Исходный дек:
5

Выполняем цифровую сортировку дека...
Отсортированный дек:
5

Поиск методом Райта
Ключ 5: НАЙДЕН на позиции 0 (просмотров: 1)
Ключ 9: не найден (просмотров: 1)
```

Граничный случай: сортировка возвращает управление сразу (менее двух элементов). Поиск обоих ключей завершается за 1 просмотр (середина — единственный элемент).

### 4.4. Тест 4: несуществующий файл

```
$ ./sort missing.txt
Чтение файла 'missing.txt'...
Ошибка: не удалось открыть файл 'missing.txt'
```

Программа корректно обрабатывает ошибку открытия файла и завершает работу с кодом `EXIT_FAILURE`.

## 4.5. Тест 5: пустой файл

```
$ ./sort empty.txt
Чтение файла 'empty.txt'...
Ошибка: файл 'empty.txt' пуст или не содержит чисел
```

Программа корректно обнаруживает пустой файл и завершает работу с кодом `EXIT_FAILURE`.

## 5. Блок-схемы алгоритма

Блок-схемы всех подпрограмм приведены в Приложении А (стр. 23).

Перечень блок-схем:

1. Главная программа (`main`)
2. Чтение из файла (`read_file`)
3. Цифровая сортировка (`radix_sort`)
4. Сортировка по разряду через дека (`counting_sort_by_deque`)
5. Добавление в голову дека (`push_front`)
6. Добавление в хвост дека (`push_back`)
7. Извлечение из головы дека (`pop_front`)
8. Извлечение из хвоста дека (`pop_back`)
9. Поиск методом Райта (`wright_search`)

## 6. Исходный код программы

Программа состоит из трёх файлов: `deque.h` — объявления структур и всех функций; `deque.c` — реализация дека, алгоритма сортировки и поиска методом Райта; `main.c` — точка входа, ввод/вывод.

### 6.1. Файл `deque.h`

```
1  #ifndef DEQUE_H
2  #define DEQUE_H
3
4  /* Узел двусвязного списка */
5  typedef struct DNode {
6      int         value; /* хранимое число */
7      struct DNode *prev; /* предыдущий узел (NULL у головы) */
8      struct DNode *next; /* следующий узел (NULL у хвоста) */
9  } DNode;
10
11 /* Дек на базе двусвязного списка:
12  * вставка и удаление с обоих концов за O(1) */
13 typedef struct {
14     DNode *front; /* голова дека */
15     DNode *back;  /* хвост дека */
16     int     size; /* число элементов */
17 } Deque;
18
19 /* Операции дека */
20 void deque_init(Deque *d);
21 int  deque_is_empty(const Deque *d);
22 int  deque_push_front(Deque *d, int value);
23 int  deque_push_back(Deque *d, int value);
24 int  deque_pop_front(Deque *d, int *out);
25 int  deque_pop_back(Deque *d, int *out);
26 void deque_free(Deque *d);
27
28 /* Цифровая сортировка (LSD Radix Sort) дека по возрастанию,
29  * поддерживает отрицательные числа */
30 void radix_sort(Deque *d);
31
32 /* Поиск методом Райта в отсортированном по возрастанию deque.
33  * Возвращает найденный узел (или NULL), в *pos позицию от 0 (или -1),
34  * в *views число просмотренных узлов */
35 const DNode *wright_search(const Deque *d, int key, int *pos, int *views);
36
37 #endif
```

## 6.2. Файл deque.c

### Операции дека

```
1 #include "deque.h"
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 #define BASE 10 /* основание системы счисления для цифровой сортировки */
6
7 /* Операции дека */
8
9 /* Инициализация пустого дека */
10 void deque_init(Deque *d)
11 {
12     d->front = NULL;
13     d->back  = NULL;
14     d->size  = 0;
15 }
16
17 /* Проверка дека на пустоту */
18 int deque_is_empty(const Deque *d)
19 {
20     return d->size == 0;
21 }
22
23 /* Добавление элемента в голову дека */
24 int deque_push_front(Deque *d, int value)
25 {
26     /* создаём новый узел */
27     DNode *node = (DNode *)malloc(sizeof(DNode));
28     if (!node) {
29         fprintf(stderr, "Ошибка: malloc вернул NULL\n");
30         return -1;
31     }
32
33     /* новый узел становится головой: prev пуст, next прежняя голова */
34     node->value = value;
35     node->prev  = NULL;
36     node->next  = d->front;
37
38     if (d->front != NULL)
39         d->front->prev = node; /* связываем прежнюю голову с новым узлом */
40     else
41         d->back = node; /* дек был пуст узел также является хвостом */
42
43     d->front = node;
44     d->size++;
```

```

45     return 0;
46 }
47
48 /* Добавление элемента в хвост дека */
49 int deque_push_back(Deque *d, int value)
50 {
51     /* создаём новый узел */
52     DNode *node = (DNode *)malloc(sizeof(DNode));
53     if (!node) {
54         fprintf(stderr, "Ошибка: malloc вернул NULL\n");
55         return -1;
56     }
57
58     /* новый узел становится хвостом: next пуст, prev прежний хвост */
59     node->value = value;
60     node->next = NULL;
61     node->prev = d->back;
62
63     if (d->back != NULL)
64         d->back->next = node; /* связываем прежний хвост с новым узлом */
65     else
66         d->front = node; /* дек был пуст узел также является головой */
67
68     d->back = node;
69     d->size++;
70     return 0;
71 }
72
73 /* Извлечение элемента из головы дека */
74 int deque_pop_front(Deque *d, int *out)
75 {
76     if (d->size == 0) {
77         fprintf(stderr, "Ошибка: дек пуст\n");
78         return -1;
79     }
80
81     /* запоминаем голову и её значение, сдвигаем голову на следующий узел */
82     DNode *node = d->front;
83     *out = node->value;
84     d->front = node->next;
85
86     if (d->front != NULL)
87         d->front->prev = NULL; /* новая голова не имеет предыдущего узла */
88     else
89         d->back = NULL; /* извлекли последний элемент дек пуст */
90
91     free(node);
92     d->size--;

```

```

93     return 0;
94 }
95
96 /* Извлечение элемента из хвоста дека */
97 int deque_pop_back(Deque *d, int *out)
98 {
99     if (d->size == 0) {
100         fprintf(stderr, "Ошибка: дек пуст\n");
101         return -1;
102     }
103
104     /* запоминаем хвост и его значение, сдвигаем хвост на предыдущий узел */
105     DNode *node = d->back;
106     *out = node->value;
107     d->back = node->prev;
108
109     if (d->back != NULL)
110         d->back->next = NULL; /* новый хвост не имеет следующего узла */
111     else
112         d->front = NULL; /* извлекли последний элемент дек пуст */
113
114     free(node);
115     d->size--;
116     return 0;
117 }
118
119 /* Освобождение всех узлов дека */
120 void deque_free(Deque *d)
121 {
122     /* проходим по списку, освобождая каждый узел */
123     DNode *cur = d->front;
124     while (cur != NULL) {
125         DNode *tmp = cur;
126         cur = cur->next;
127         free(tmp);
128     }
129     d->front = NULL;
130     d->back = NULL;
131     d->size = 0;
132 }

```

## Цифровая сортировка

```

1 /* Цифровая сортировка дека */
2
3 /* Поиск максимального значения в непустом deque
4  * определяет число разрядов (проходов сортировки) */

```



```

5 static int find_max(const Deque *d)
6 {
7     int max = d->front->value;
8     const DNode *cur = d->front->next;
9     while (cur != NULL) {
10         if (cur->value > max)
11             max = cur->value;
12         cur = cur->next;
13     }
14     return max;
15 }
16
17 /* Один проход сортировки по разряду exp (1, 10, 100, ...).
18  * Вместо массивов используются 10 деков-корзин по одному на цифру от 0 до 9 */
19 static void counting_sort_by_deque(Deque *d, int exp)
20 {
21     Deque buckets[BASE];
22     int value;
23
24     /* инициализируем корзины */
25     for (int j = 0; j < BASE; j++)
26         deque_init(&buckets[j]);
27
28     /* фаза распределения: извлекаем числа из дека
29     * и раскладываем по корзинам согласно текущей цифре */
30     while (!deque_is_empty(d)) {
31         deque_pop_front(d, &value);
32         int digit = (value / exp) % BASE; /* цифра текущего разряда */
33         deque_push_back(&buckets[digit], value);
34     }
35
36     /* фаза сбора: возвращаем числа в дек, обходя корзины от 0 до 9.
37     * Порядок FIFO (push_back/pop_front) обеспечивает устойчивость */
38     for (int j = 0; j < BASE; j++) {
39         while (!deque_is_empty(&buckets[j])) {
40             deque_pop_front(&buckets[j], &value);
41             deque_push_back(d, value);
42         }
43     }
44 }
45
46 /* Цифровая сортировка дека из неотрицательных чисел */
47 static void radix_sort_nonneg(Deque *d)
48 {
49     if (d->size <= 1) return; /* нечего сортировать */
50
51     /* проходы по разрядам от младшего к старшему,
52     * пока в максимуме остаются цифры */

```

```

53     int max_val = find_max(d);
54     for (int exp = 1; max_val / exp > 0; exp *= BASE)
55         counting_sort_by_deque(d, exp);
56 }
57
58 /* Цифровая сортировка дека с поддержкой отрицательных чисел */
59 void radix_sort(Deque *d)
60 {
61     if (d->size <= 1) return; /* нечего сортировать */
62
63     /* вспомогательные дека: модули отрицательных и положительных числа */
64     Deque neg, pos;
65     deque_init(&neg);
66     deque_init(&pos);
67
68     /* разделяем исходный дек на два по знаку */
69     int value;
70     while (!deque_is_empty(d)) {
71         deque_pop_front(d, &value);
72         if (value < 0)
73             deque_push_back(&neg, -value); /* храним модуль */
74         else
75             deque_push_back(&pos, value);
76     }
77
78     /* сортируем обе части как неотрицательные */
79     radix_sort_nonneg(&neg);
80     radix_sort_nonneg(&pos);
81
82     /* собираем результат: отрицательные идут первыми.
83     * Дек neg отсортирован по модулю, поэтому берём с хвоста:
84     * наибольший модуль даёт наименьшее отрицательное число */
85     while (!deque_is_empty(&neg)) {
86         deque_pop_back(&neg, &value);
87         deque_push_back(d, -value); /* возвращаем знак */
88     }
89
90     /* положительные добавляем в исходном (возрастающем) порядке */
91     while (!deque_is_empty(&pos)) {
92         deque_pop_front(&pos, &value);
93         deque_push_back(d, value);
94     }
95 }

```

## Поиск методом Райта

```

1 /* Поиск методом Райта */

```

```

2
3  /* Поиск в отсортированном deque начинается со среднего элемента:
4  * сравнение с серединой определяет направление движения (prev или next),
5  * что в среднем вдвое сокращает число просмотров по сравнению
6  * с линейным поиском от головы */
7  const DNode *wright_search(const Deque *d, int key, int *pos, int *views)
8  {
9      *views = 0;
10     *pos = -1;
11     if (d->size == 0)
12         return NULL; /* в пустом deque искать нечего */
13
14     /* шаг 1: выходим на средний элемент списка */
15     const DNode *cur = d->front;
16     int idx = 0;
17     int mid = d->size / 2;
18     while (idx < mid) {
19         cur = cur->next;
20         idx++;
21     }
22
23     /* шаг 2: сравниваем ключ со средним элементом */
24     (*views)++;
25     if (key == cur->value) {
26         *pos = idx;
27         return cur;
28     }
29
30     /* шаг 3: двигаемся влево или вправо от середины,
31     * пока значения загораживают ключ */
32     if (key < cur->value) {
33         /* ключ меньше середины идём к голове по prev */
34         cur = cur->prev;
35         idx--;
36         while (cur != NULL && cur->value > key) {
37             (*views)++;
38             cur = cur->prev;
39             idx--;
40         }
41     } else {
42         /* ключ больше середины идём к хвосту по next */
43         cur = cur->next;
44         idx++;
45         while (cur != NULL && cur->value < key) {
46             (*views)++;
47             cur = cur->next;
48             idx++;
49         }

```

```

50     }
51
52     /* шаг 4: проверяем узел, на котором остановились */
53     if (cur != NULL) {
54         (*views)++;
55         if (cur->value == key) {
56             *pos = idx;
57             return cur;
58         }
59     }
60     return NULL; /* ключ отсутствует в деке */
61 }

```

### 6.3. Файл main.c

```

1  #include "deque.h"
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  /* Чтение чисел из файла в дек.
6   * Возвращает число прочитанных элементов или -1 при ошибке */
7  static int read_file(const char *filename, Deque *d)
8  {
9      FILE *fp = fopen(filename, "r");
10     if (!fp) {
11         fprintf(stderr, "Ошибка: не удалось открыть файл '%s'\n", filename);
12         return -1;
13     }
14
15     /* читаем числа по одному и добавляем в хвост дека */
16     int tmp;
17     while (fscanf(fp, "%d", &tmp) == 1) {
18         if (deque_push_back(d, tmp) != 0) {
19             deque_free(d);
20             fclose(fp);
21             return -1;
22         }
23     }
24
25     fclose(fp);
26     if (deque_is_empty(d)) {
27         fprintf(stderr, "Ошибка: файл '%s' пуст или не содержит чисел\n", filename);
28         return -1;
29     }
30     return d->size;
31 }
32

```

```

33  /* Вывод содержимого дека от головы к хвосту */
34  static void deque_print(const Deque *d)
35  {
36      const DNode *cur = d->front;
37      while (cur != NULL) {
38          printf("%d", cur->value);
39          if (cur->next != NULL)
40              printf(" ");
41          cur = cur->next;
42      }
43      printf("\n");
44  }
45
46  /* Поиск ключа методом Райта с выводом результата */
47  static void search_demo(const Deque *d, int key)
48  {
49      int pos, views;
50      const DNode *node = wright_search(d, key, &pos, &views);
51      if (node != NULL)
52          printf("Ключ %d: НАЙДЕН на позиции %d (просмотров: %d)\n",
53                key, pos, views);
54      else
55          printf("Ключ %d: не найден (просмотров: %d)\n", key, views);
56  }
57
58  int main(int argc, char *argv[])
59  {
60      if (argc < 2) {
61          fprintf(stderr, "Использование: %s <файл> [ключ поиска ...]\n", argv[0]);
62          return EXIT_FAILURE;
63      }
64      const char *filename = argv[1];
65
66      /* читаем входные данные в дек */
67      printf("\nЧтение файла '%s'...\n", filename);
68      Deque d;
69      deque_init(&d);
70      int n = read_file(filename, &d);
71      if (n < 0) return EXIT_FAILURE;
72      const char *form = (n % 100 >= 11 && n % 100 <= 19) ? "чисел"
73                      : (n % 10 == 1)                  ? "число"
74                      : (n % 10 >= 2 && n % 10 <= 4)    ? "числа"
75                      : "чисел";
76      printf("Прочитано %d %s.\n", n, form);
77
78      printf("\nИсходный дек:\n");
79      deque_print(&d);
80

```

```

81  /* цифровая сортировка дека */
82  printf("\nВыполняем цифровую сортировку дека...\n");
83  radix_sort(&d);
84
85  printf("Отсортированный дек:\n");
86  deque_print(&d);
87
88  /* поиск методом Райта: ключи из аргументов или с клавиатуры */
89  printf("\n Поиск методом Райта \n");
90  if (argc > 2) {
91      for (int i = 2; i < argc; i++)
92          search_demo(&d, atoi(argv[i]));
93  } else {
94      printf("Введите ключ для поиска: ");
95      int key;
96      if (scanf("%d", &key) == 1)
97          search_demo(&d, key);
98      else
99          fprintf(stderr, "Ошибка: ключ не прочитан\n");
100  }
101  printf("\n");
102
103  /* освобождаем память дека */
104  deque_free(&d);
105  return EXIT_SUCCESS;
106 }

```

## Заключение

В ходе выполнения лабораторной работы была разработана программа на языке C, реализующая цифровую сортировку LSD целых чисел (включая отрицательные), считываемых из файла, с использованием **дека на основе двусвязного списка** в качестве основной и вспомогательной структуры данных, а также **поиск методом Райта** в отсортированном деке.

Программа организована в три файла: `deque.h` объявляет структуры и интерфейс; `deque.c` реализует дек, сортировку и поиск; `main.c` содержит точку входа и ввод/вывод.

Поддержка отрицательных чисел реализована через разделение дека на два вспомогательных (модули отрицательных и положительных), их независимую сортировку и сборку результата: из **neg** элементы извлекаются с хвоста (чтобы сохранить порядок), из **pos** — с головы.

Поиск методом Райта использует двусвязность списка: стартует со среднего элемента и движется по **prev** или **next** в зависимости от соотношения ключа со средним значением.

Результаты тестирования подтверждают корректность сортировки для всех рассмотренных случаев — стандартного, с отрицательными числами, с одним элементом — а также правильную обработку ошибочных ситуаций.

## Приложение А. Блок-схемы алгоритма



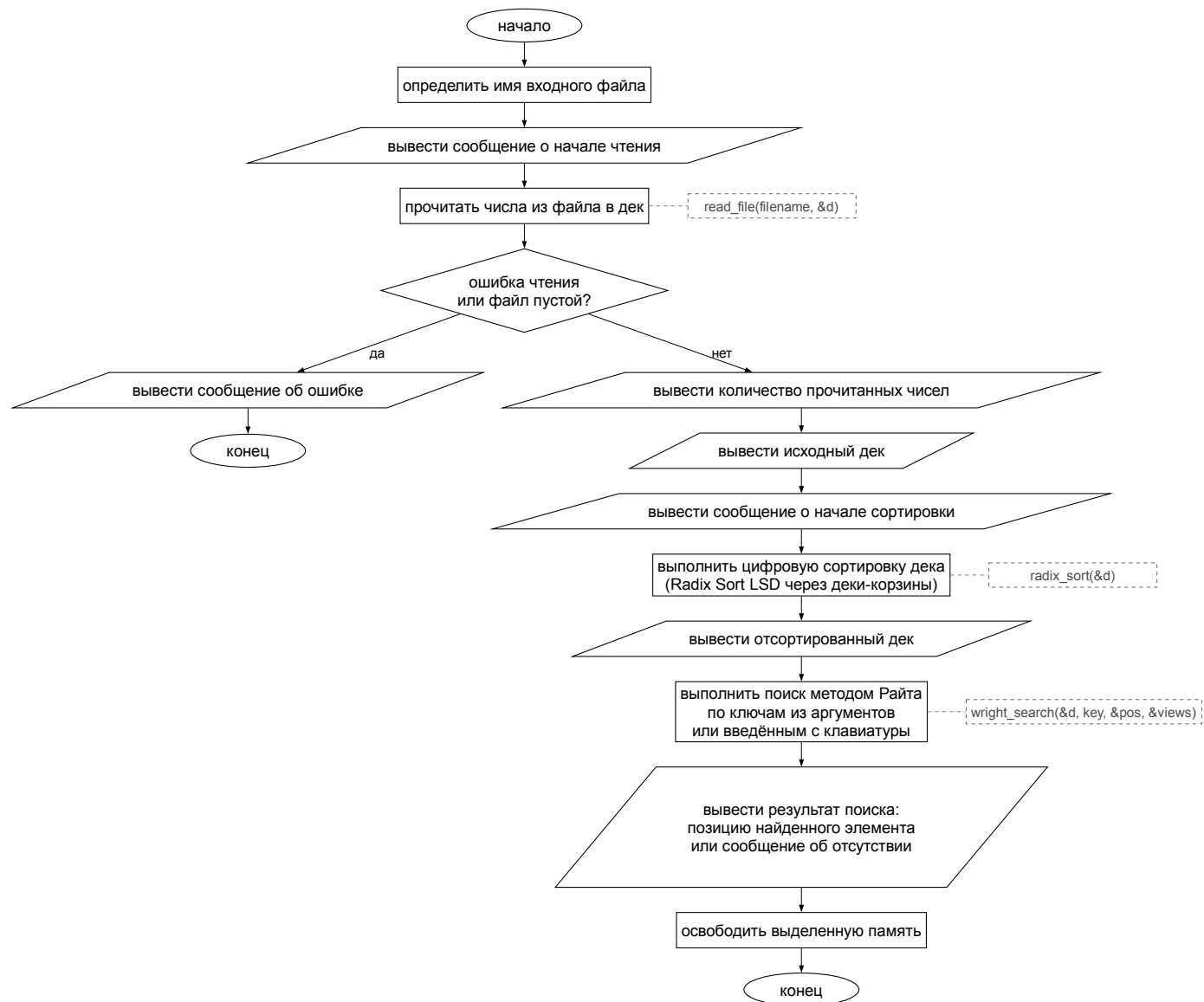


Рис. 1 – Главная программа (main)

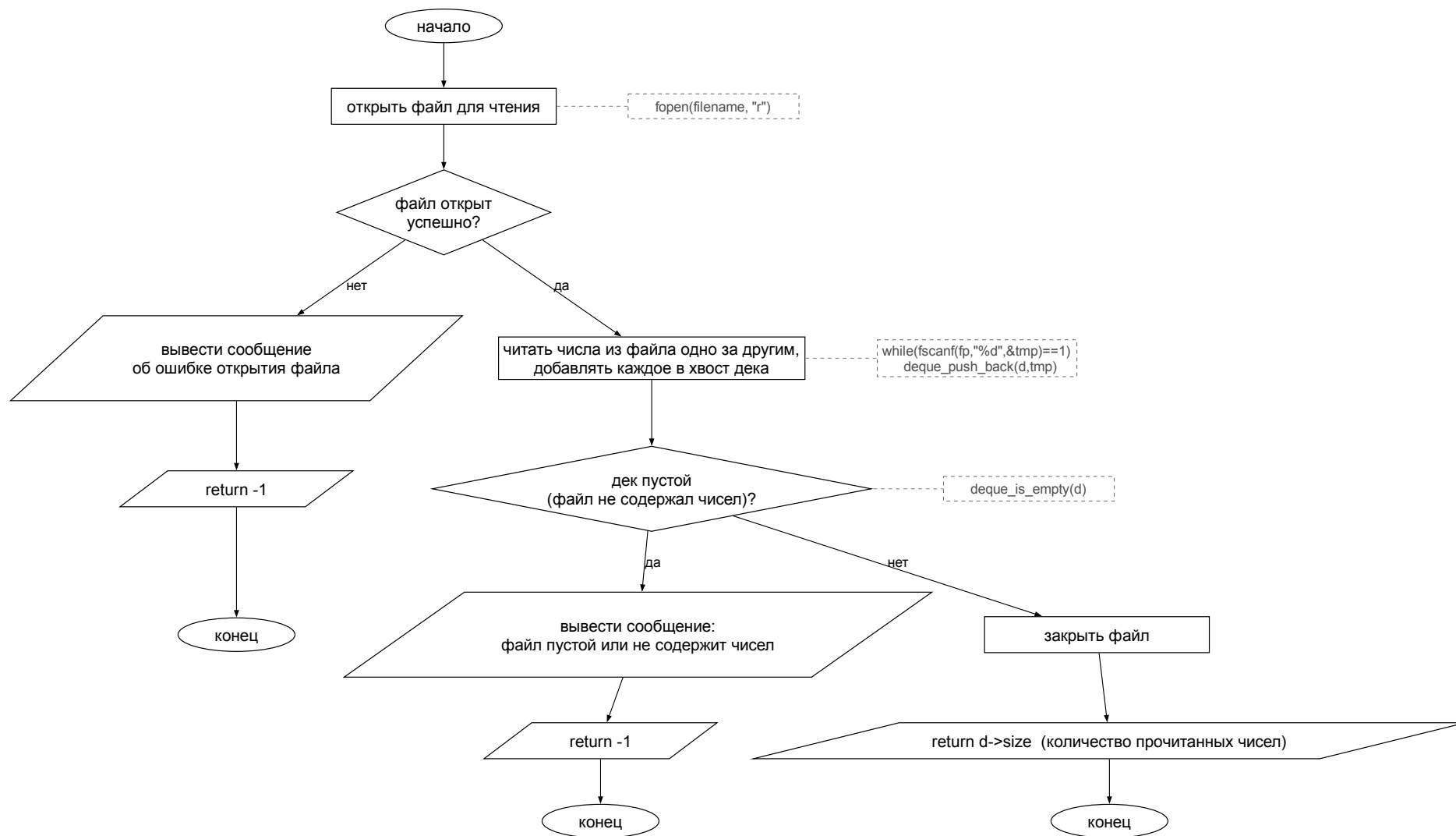


Рис. 2 – Чтение из файла (read\_file)

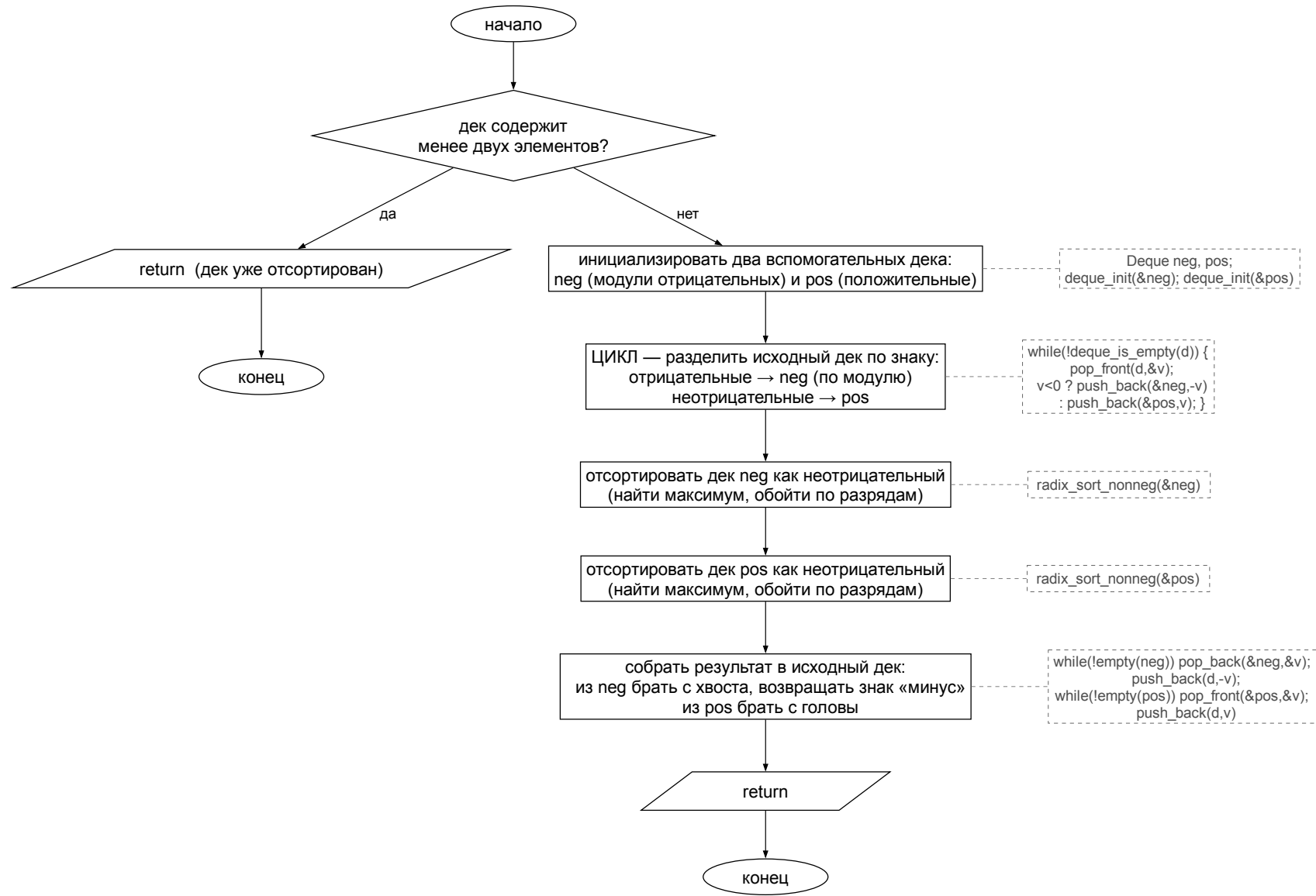


Рис. 3 – Цифровая сортировка (radix\_sort)

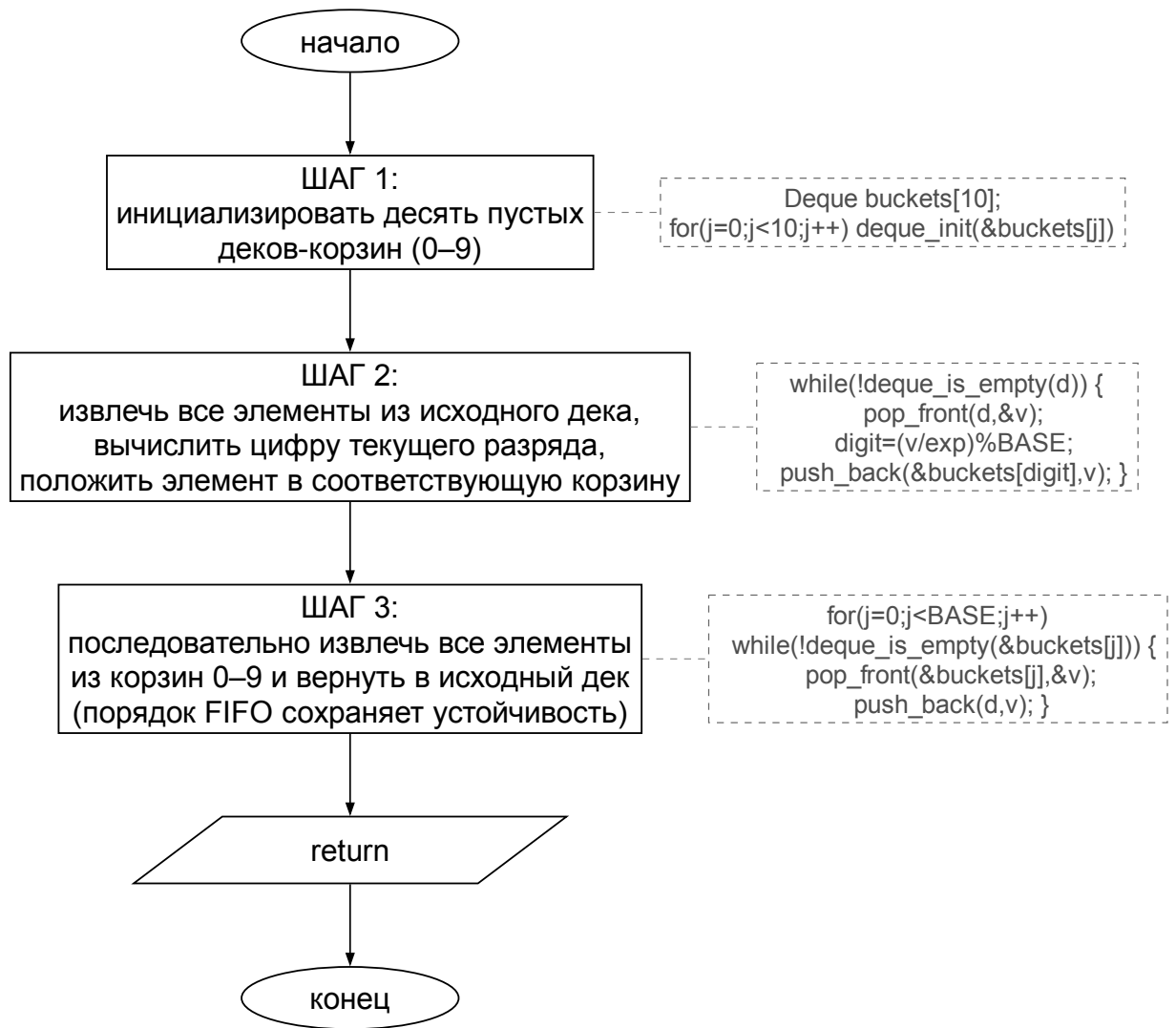


Рис. 4 – Сортировка по разряду через деки (counting\_sort\_by\_deque)

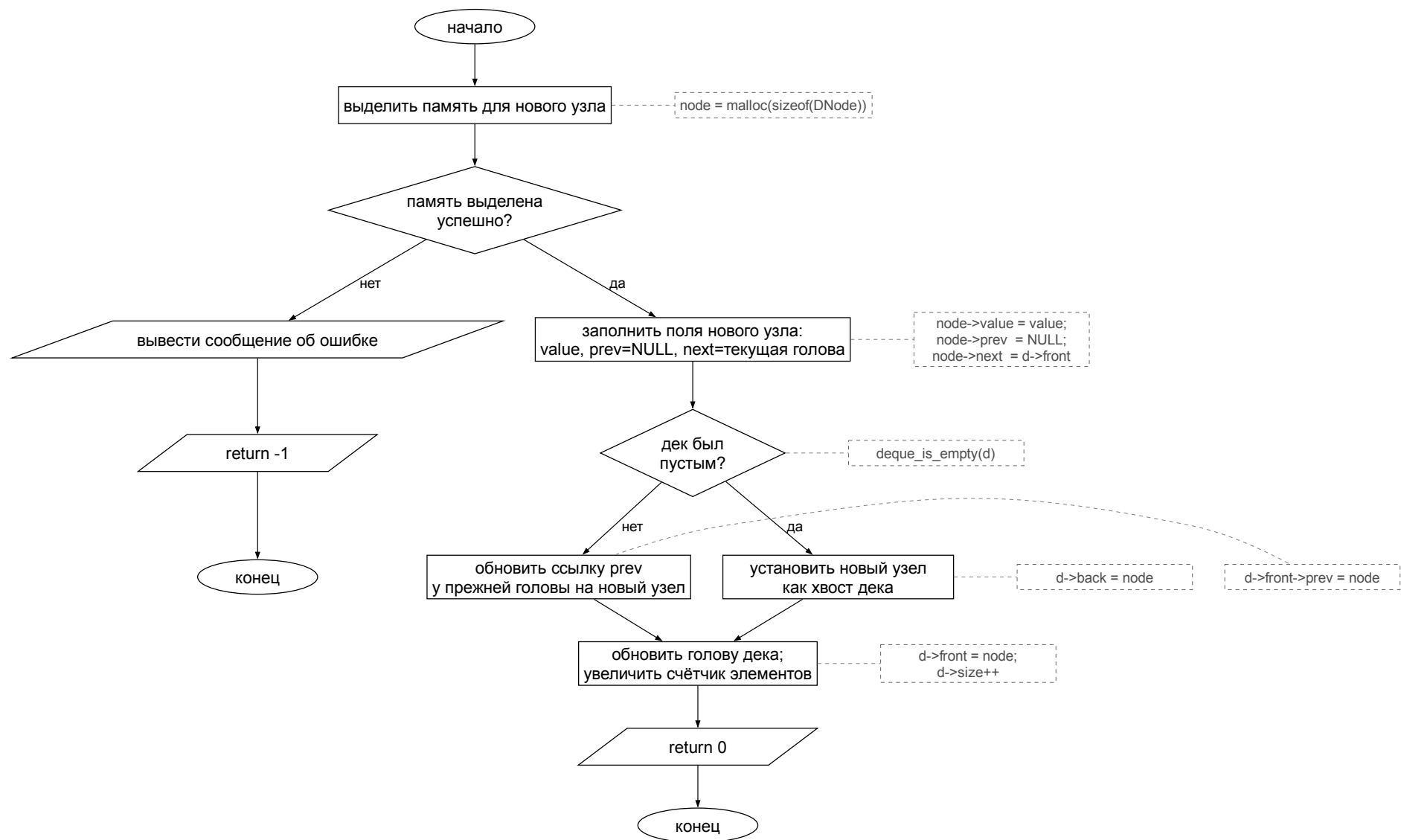


Рис. 5 – Добавление в голову дека (push\_front)

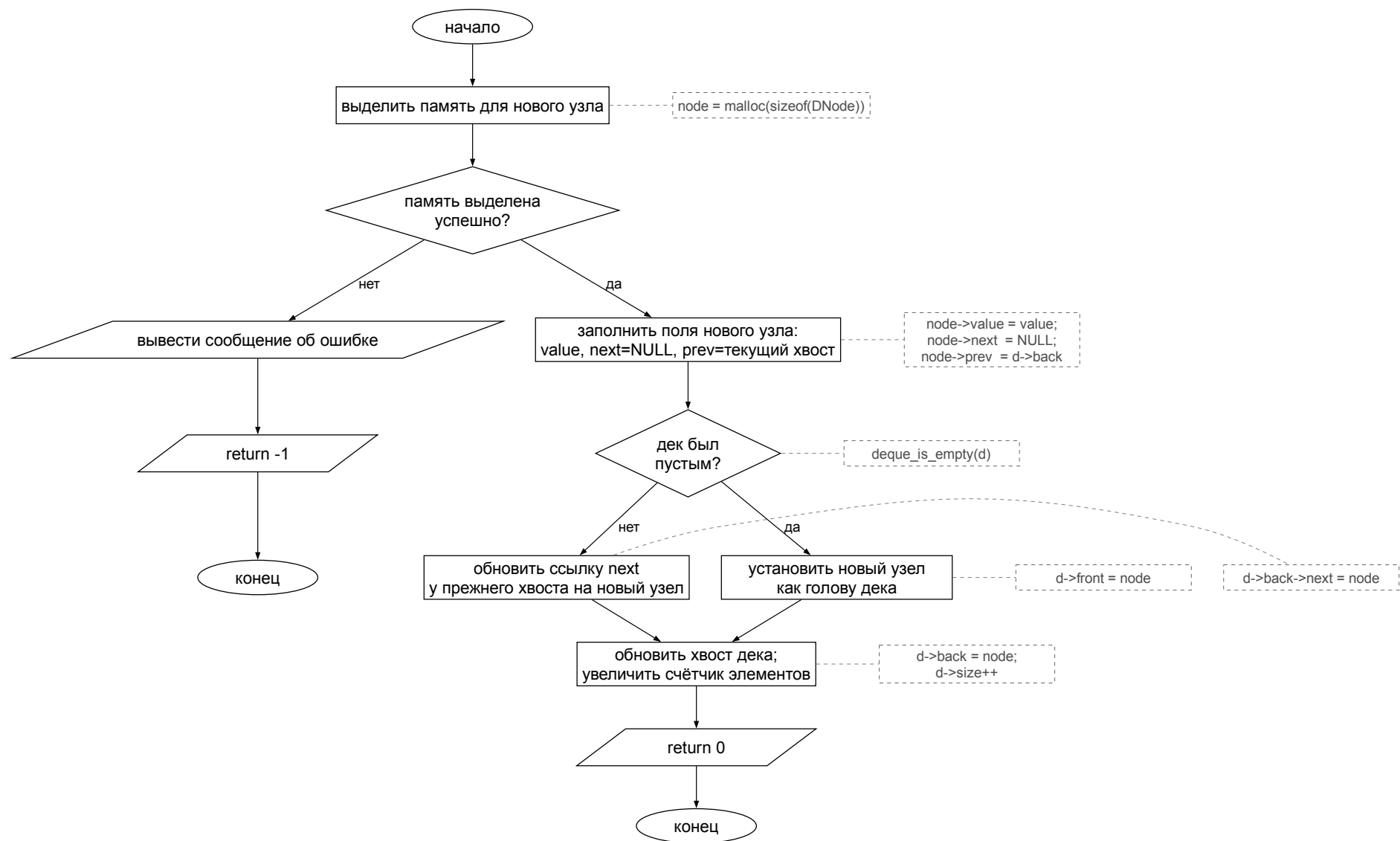


Рис. 6 – Добавление в хвост дека (push\_back)

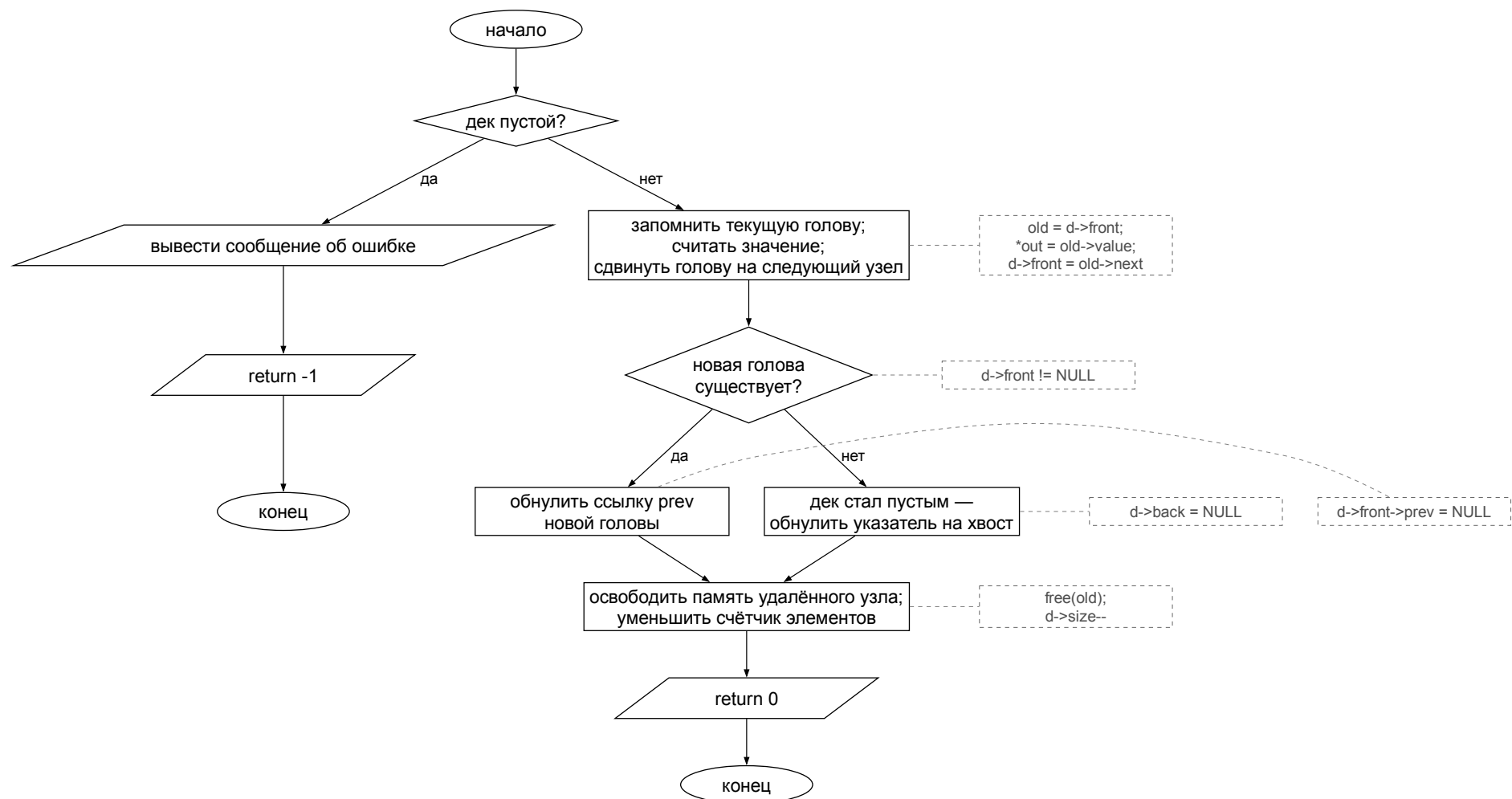


Рис. 7 – Извлечение из головы дека (pop\_front)

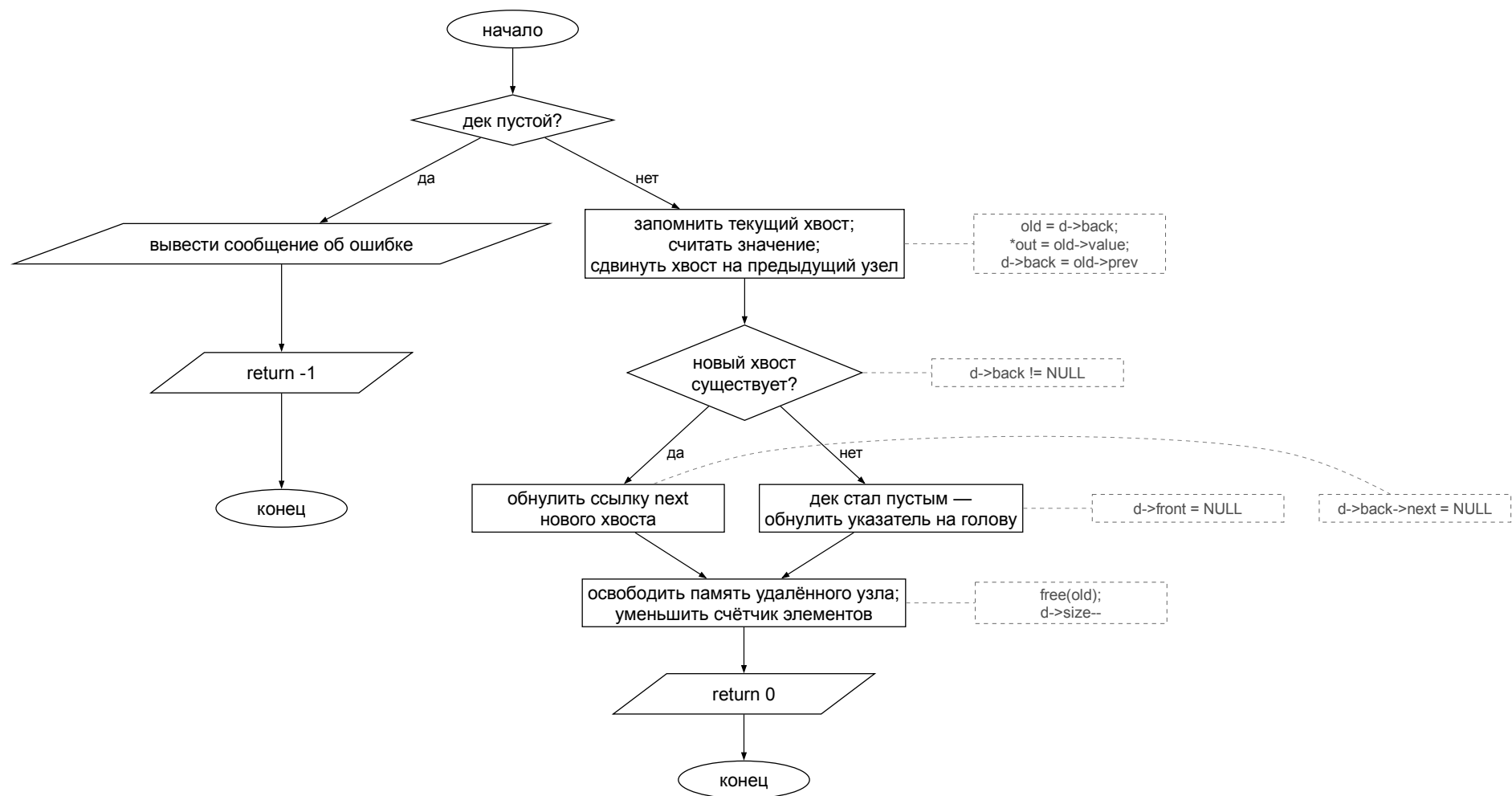


Рис. 8 – Извлечение из хвоста дека (pop\_back)



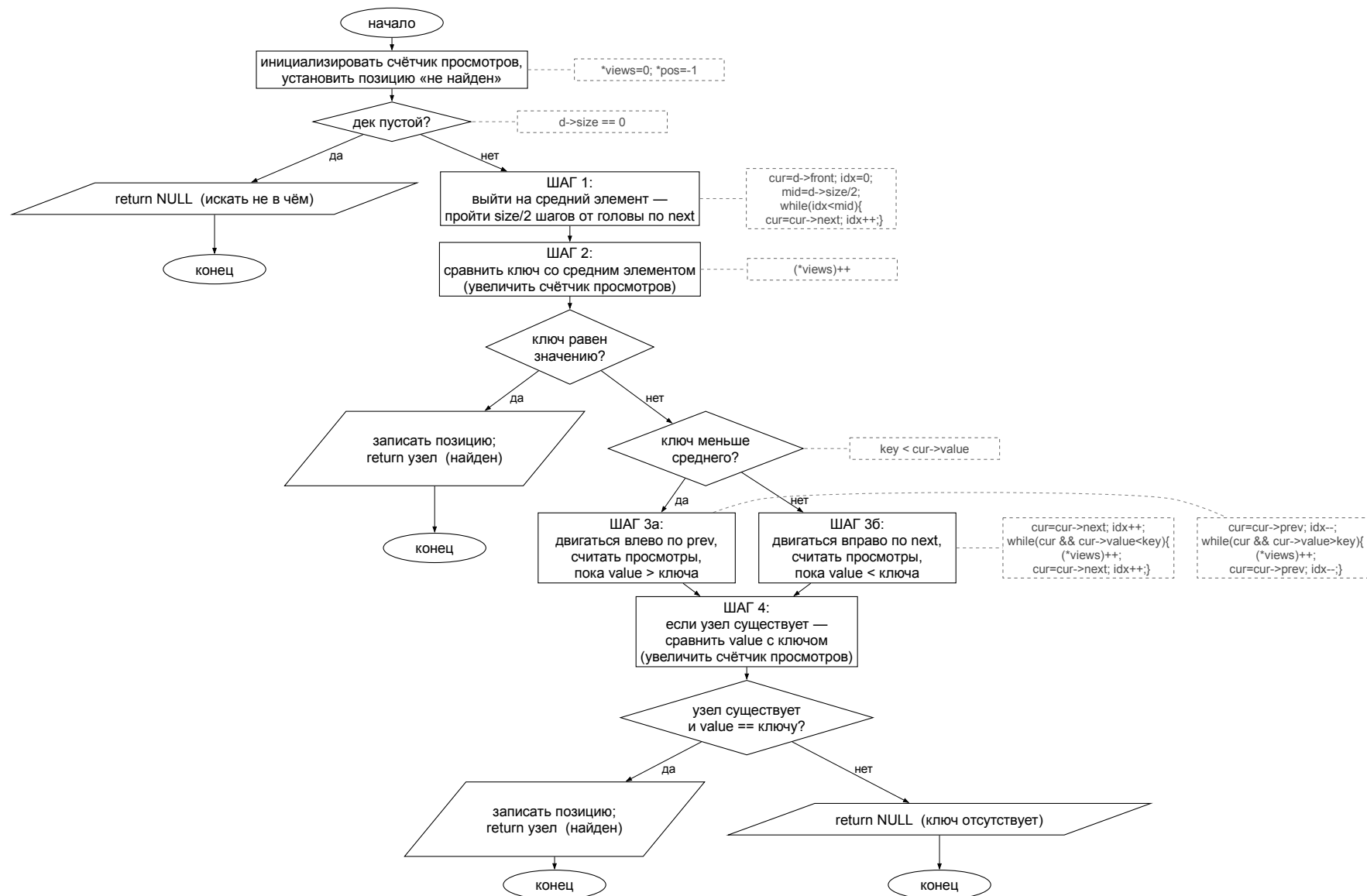


Рис. 9 – Поиск методом Райта (wright\_search)